

AI Data Science Workbench

A governed, reproducible machine-learning platform

Machine Learning (23BA045E) · Amrita School of Business

Project report — technical and architectural detail

Generated 2026-08-03

Contents

Contents.....	2
1. What this project is.....	3
Three ways in, one pipeline	3
2. The pipeline	3
The doctrines it enforces	3
3. Architecture	4
Layout	4
Two workspace layouts.....	4
The Python bridge.....	4
4. The web application.....	4
Design system	5
5. Results.....	5
6. Engineering quality	5
What is verified	5
Defects found by audit.....	5
7. Known limits	6
8. Summary	6

1. What this project is

The AI Data Science Workbench is a machine-learning platform that takes a tabular dataset from raw CSV to a defended, explainable model — and records every decision it made along the way. It was built for a graded MBA machine-learning project (California Housing regression), then generalised so that any uploaded CSV runs through the same governed pipeline.

The distinguishing idea is governance. Most teaching pipelines train a model and print a score. This one refuses to train a model that has not been approved, refuses to touch the test split more than once, records who approved what and why, and writes an audit trail and a lineage graph alongside the metrics. The claim it is designed to support is not "the model scores X" but "here is exactly how the model was produced, and here is the evidence".

Three ways in, one pipeline

The same Python pipeline is driven by three interfaces, which matters architecturally: none of them owns the logic.

Interface	What it is	Role
main.py	Command-line interface	The authoritative pipeline. Every stage is a subcommand.
app/	Streamlit GUI, 11 pages	Read-only dashboard over the artifacts the CLI writes.
webapp/	Next.js 15 web application	Upload-first product UI: drop a CSV, watch it process, walk nine stages of evidence.

2. The pipeline

Eleven ordered stages, each with declared inputs, outputs and prerequisites. The orchestrator refuses to run a stage whose prerequisites have not produced their artifacts, so the chain cannot silently run out of order.

#	Stage	What it produces
1	ingest	Raw CSV pulled into the workspace, profiled and validated
2	preprocess	Cleaned dataset, 70/15/15 split, train-only transforms fitted
3	understand	Data-understanding tables
4	eda	Exploratory analysis, figure gallery, analytical summary
5	features	Feature proposal across five governed groups, then execution
6	propose	Model proposal: scope, rationale, candidate catalogue
7	approve	The governance gate — nothing unapproved may train
8	train	Every approved model, versioned into the registry
9	evaluate	Validation comparison, champion promotion, one test measurement
10	explain	Standardized coefficients, top drivers, executive briefing
11	report	Self-contained HTML project report

The doctrines it enforces

- Split early. The data is split before anything is fitted, so no statistic can be learned from validation or test.
- Train-only fitting. Every imputation median, scaling parameter and encoder category comes from the training split, and each is persisted to disk so the claim is checkable rather than asserted.
- The test split is read once. Only the champion is scored against it, only at the end, only for the final number.
- Approval gates training. A model absent from the approved catalogue cannot be trained; an unapproved feature cannot reach the modelling matrix.
- Nothing is deleted silently. Outliers are reported, never dropped; every governed decision is written to an append-only audit log.

3. Architecture

Layout

<code>main.py</code>	CLI entry point, one subcommand per stage
<code>config/</code>	paths, constants, dataset registry, active project
<code>src/</code>	78 modules, ~9,800 lines
<code>automl/</code>	detect task+target, register uploads, auto-report
<code>preprocessing/</code>	cleaning, splitting, train-only transformers
<code>feature_engineering/</code>	proposal, approval, execution
<code>model_training/</code>	catalogue, trainer, metrics
<code>model_evaluation/</code>	comparison, champion selection, diagnostics
<code>explainability/</code>	coefficients, drivers, briefings
<code>governance/</code>	audit log, lineage, approvals, decisions
<code>reporting/</code>	HTML report generator
<code>workflow/</code>	orchestrator and stage registry
<code>webapi/bridge.py</code>	the only Node-to-Python boundary
<code>webapp/</code>	Next.js app, 63 files, ~7,700 lines
<code>app/</code>	Streamlit GUI, 11 pages

Two workspace layouts

The graded California Housing project keeps a flat layout at the repository root, because its model registry stores absolute paths and its report already lives there. Every other project is isolated under `workspaces/<slug>/`. Which layout applies is decided by the `WORKBENCH_PROJECT` environment variable, read by `config/paths.py` at import time — a detail that has real consequences, since setting the variable after importing a path-dependent module is a silent no-op.

The Python bridge

The web application never imports Python. It spawns `webapi/bridge.py`, which guarantees that `stdout` carries JSON and nothing else — all logging goes to `stderr`. The `detect` subcommand returns a single object; `analyze` streams one JSON line per stage so the browser can show real progress. The consequence is that the web app is localhost-only by design: it runs next to the Python installation, not in a datacentre.

4. The web application

The product UI is upload-first. The landing page is the drop zone itself, not a page containing one. After a file is scanned, the user confirms what to predict, then watches five real stages complete — each corresponding to actual Python work, not a timer.

Step shown	What actually runs
Uploading	The file is written to disk
Scanning document	Column profiling, task and target detection
Extracting data	Project registration and preparation, including the split
Running analytics	Training every approved model, then evaluation
Generating insights	Auto-report, driver table, executive briefing, HTML report

The result is a nine-stage journey — Dashboard, Data, EDA, Features, Modeling, Evaluation, Explainability, Governance, Reports — each backed by artifacts the pipeline actually wrote. A stage with no artifacts is shown dimmed and labelled, never hidden: hiding it would make the pipeline look shorter than it is.

Design system

- One brand accent (a deep blue) with neutrals dominating; semantic green/amber/red reserved for data status, never for interface chrome.
- Full light and dark themes, with an anti-flash script so a stored preference applies before first paint.
- All text verified against WCAG AA contrast in both themes, computed rather than eyeballed.
- Charts restricted to bar, line and donut, each with a mandatory plain-language caption.

5. Results

Two reference projects ship with the platform. Both figures below are test-split measurements, taken once.

Project	Task	Champion	Test performance
California Housing	Regression	Ridge v001	R^2 0.667 · RMSE 66,681 · MAE 48,731
Customer Churn (Telco)	Classification	LogisticRegression v001	ROC-AUC 0.845 · Accuracy 0.811 · F1 0.598

A further eleven projects have been created through the upload flow, covering regression and classification on unrelated datasets — evidence that the pipeline generalises beyond the two it was written for.

6. Engineering quality

What is verified

- 69 Python tests covering schema constants, split determinism, train-only fitting, the approval gate, orchestrator dependency resolution and metric selection.
- TypeScript strict mode, a production build, and a unit check, all run by a single npm run verify.
- Every one of the 27 stage pages across three projects returns HTTP 200.

Defects found by audit

A structured audit of the codebase found eight defects that no test had caught. The most serious produced wrong results silently rather than failing:

Severity	Defect
High	A NaN metric could win champion selection, promoting an unevaluated model over a better one
High	A constant target crashed inside sklearn instead of being refused with a clear message
Medium-High	A positive-class flag silently binarised a multiclass target while metadata still claimed multiclass
Medium	A non-UTF-8 CSV crashed the detect step with no JSON output, breaking its own contract
Medium	An all-null column reached the estimator as NaN
Medium	The stage rail never marked the current stage, for sighted or assistive users
Medium	Dark-theme tertiary text failed WCAG AA contrast on every label and caption
Low	Two dead components

All are fixed. The NaN defect is worth singling out because it is ordering-dependent — a NaN in the middle of the candidate list selects correctly, a NaN first does not — which is precisely why it survived every test run. It is now covered by 20 regression tests, proven to fail against the original code before being restored.

7. Known limits

- The web application is localhost-only by design: it spawns a local Python process and cannot be hosted as-is.
- PDF export shells out to a locally installed Chrome or Edge. Where neither exists, the interface says so and offers the HTML instead.
- The executive briefing for the graded housing project is hand-written prose specific to California house prices. Uploaded datasets receive a deliberately generic briefing that states the metrics and drivers and makes no claim about what the target means.
- Two front-ends now exist over the same artifacts. They do not conflict, but they will drift unless consolidated.
- Model performance reflects the datasets supplied; the housing R^2 of 0.667 is a property of that data and a linear model, not a target that was tuned toward.

8. Summary

The workbench is a working, governed ML platform of roughly 17,500 lines across Python and TypeScript, exercised by 69 tests and thirteen real projects. Its value is less in any single model than in the discipline it enforces: the test split is measured once, every fitted statistic is provably train-only, nothing untracked reaches a model, and every number on screen traces back to a file on disk that can be opened and checked.